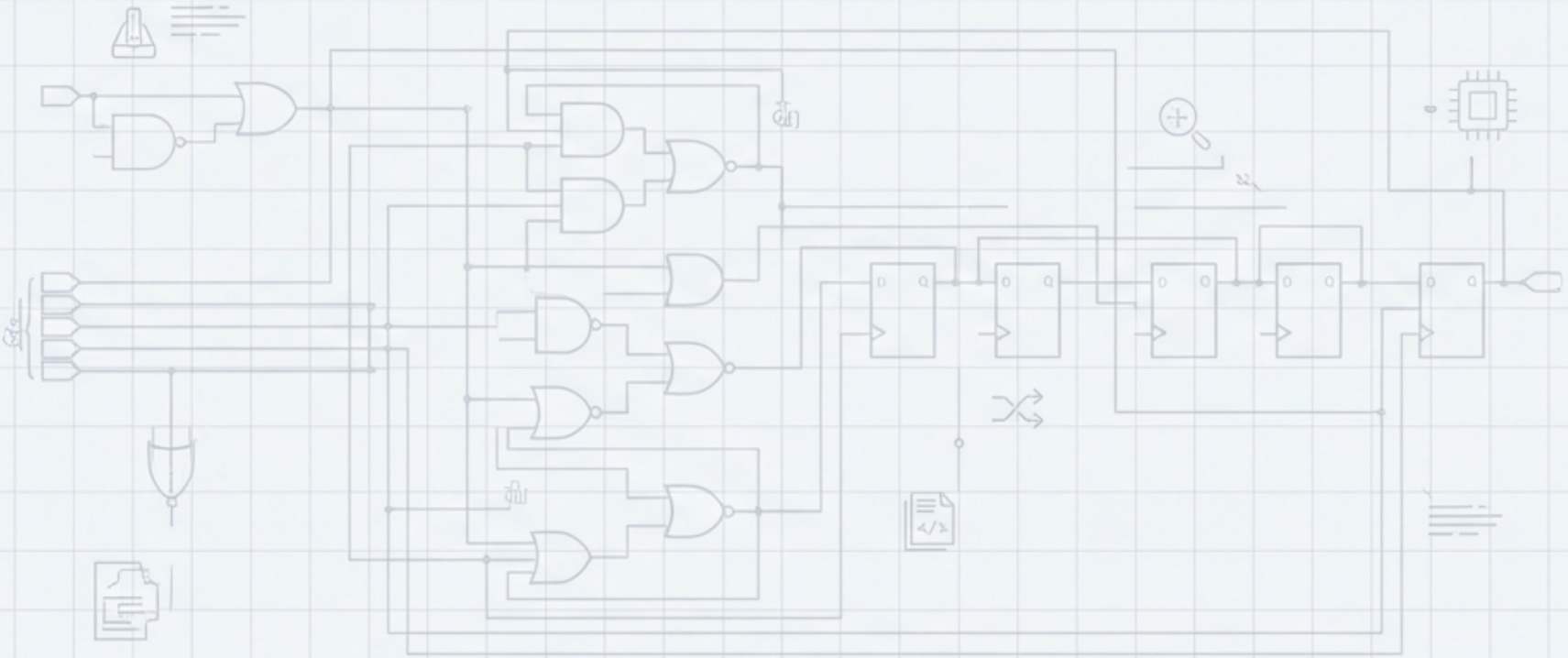
AutoChip - Lab 2

Validating LLM-Driven Verilog Generation with GPT-4o-mini



Dhirajzen Bagawath Geetha Kumaravel | db5309 | September 19, 2025

GPT-4o-mini Generated Functionally Correct Verilog for All Five Designs on the First Attempt.

5/5 DESIGNS PASSED

All five hardware modules passed simulation without manual intervention.

0 ITERATIONS REQUIRED

Correct code was generated on the initial attempt, rendering the AutoChip feedback loop unnecessary.

0 PROMPT TUNING

The initial system and module prompts were sufficient for success across all test cases.

This report details the methodology and consistent, high-quality results achieved using the AutoChip framework.

The Experimental Framework: A Rigorous AutoChip Environment

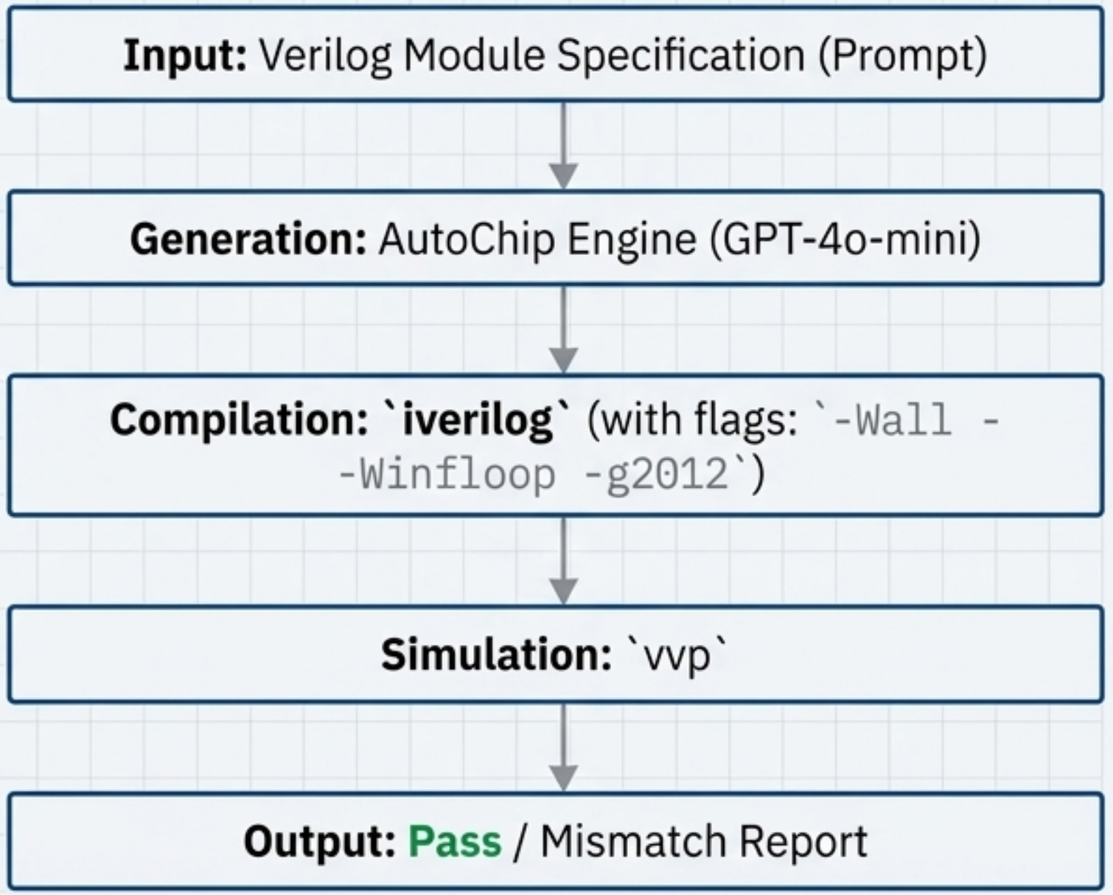
Tool Stack

LLM Model: GPT-4o-mini (via ChatGPT family)

Interface: Google Colab with AutoChip scripts

Key Parameters: `iterations_max: 10`, `num_candidates: 5`

Simulation & Verification Workflow



The Guiding Instruction: A System Prompt Engineered for Verilog

Sets the context and role.

```
You are an autocomplete engine for Verilog code. Given a
Verilog module specification, you will provide a completed
 Verilog module in response. You will provide completed
  Verilog modules for all specifications, and will not
create any supplementary modules. Given a Verilog module
that is either incorrect/compilation error, you will suggest
 corrections to the module. You will not refuse. Format
 your response as Verilog code containing the end to end
 corrected module.
```

A critical instruction to prevent conversational avoidance and force a code-based response.

Ensures machine-parsable output, avoiding extraneous text.

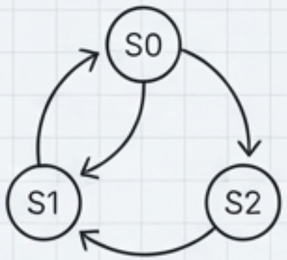
The Gauntlet: Five Diverse Hardware Design Challenges

PWM Generator

Type: Counter & Timing Logic

Test: **~2400** cycles

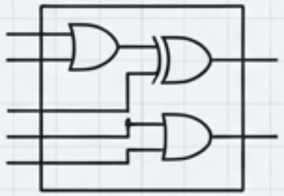
Sequence Detector



Type: Finite State Machine (FSM)

Test: **12** specific input sequences

Binary-to-BCD



Type: Combinational Arithmetic

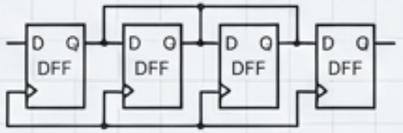
Test: **32** exhaustive test cases

Dice Roller

Type: Randomized Logic & State

Test: **4000** random trials

Shift Register



Type: Sequential Storage

Test: **7** cycles of shifting

Case Study 1: Generating Precise Counter and PWM Logic

`pwm.sv`

Challenge

Create a Pulse-Width Modulation generator, a common module requiring precise timing and counter logic.

Verification

Testbench: `pwm_tb.sv`

Samples: Tested across ~2400 simulation cycles to verify duty cycle and frequency.

Result

Candidates: 5 generated.

0

Mismatches: 0 in 4 candidates, 1 in 1 candidate.

Iterations Needed: 0

Case Study 2: Implementing a Finite State Machine

`sequence_detector.v`

Challenge

Design a Finite State Machine to detect a specific binary sequence, a core task in control logic design.

Verification

Testbench: `sequence_detector_tb.v`

Samples: Tested against 12 input samples covering success and failure cases.

Result

Candidates: 5 generated.

0

Mismatches: 0 across all 5 candidates.

Iterations Needed: 0

Case Study 3: Synthesizing Combinational Arithmetic Logic

`module `binary_to_bcd.v``

Challenge

Convert a binary input into a Binary-Coded Decimal output, a fundamental data encoding task.

Verification

- **Testbench:** `binary_to_bcd_tb.v`
- **Samples:** Tested exhaustively over 32 test cases.

Result

Candidates: 5 generated.

Mismatches: 0 across all 5 candidates. The generated RTL was noted as being particularly concise.

Iterations Needed: 0

Case Study 4: Stress-Testing Randomized Logic and State

`module `dice_roller.v``

Challenge

Create a module that simulates rolling dice, requiring an understanding of randomized inputs and stateful behavior.

Verification

Testbench: `dice_roller_tb.v`

Samples: Stress-tested across 4000 random trials to ensure robustness.

Result

Candidates: 5 generated.

0

Mismatches: 0 across all 5 candidates.

Iterations Needed: 0

Case Study 5: Verifying Foundational Sequential Storage

`module `shift_register.v``

Challenge

Implement a basic shift register, a cornerstone of sequential and data-path logic.

Verification

- Testbench: `shift_register_tb.v`
- Samples: Tested over 7 clock cycles to observe correct bit-shifting behavior.

Result

Candidates: 5 generated.

Mismatches: **0** across all 5 candidates.

Iterations Needed: **0**

A Clear Pattern of First-Attempt Success Across All Designs

Design	Design Type	Samples Tested	Mismatches	Iterations Needed
PWM Generator	Counter/PWM	~2400	**0** (4/5), **1** (1/5) Note is maimateri only, there mors mismatches.	**0**
Sequence Detector	FSM	12	✓ **0**	**0**
Binary-to-BCD	Arithmetic	32	✓ **0**	**0**
Dice Roller	Randomized	4000	✓ **0**	**0**
Shift Register	Sequential	7	✓ **0**	**0**

The AutoChip refinement loop was not invoked for any of the five designs, demonstrating the high initial quality of the LLM-generated Verilog.

Conclusion: A Successful Validation of LLM-Driven Hardware Generation

High Efficacy

The AutoChip workflow, powered by GPT-4o-mini, demonstrated the ability to generate correct-by-construction Verilog for a diverse set of standard digital logic problems.

Exceptional Efficiency

Convergence at '**iteration 0**' was the consistent outcome, highlighting the potential for significant acceleration in routine design tasks.

Prompting is Key

The success relied on a well-defined system prompt that constrained the LLM to produce structured, usable code without refusal.

This experiment serves as strong evidence for the viability of LLM-based automation in the hardware design and verification process.